

Manual de configuración

Single Sign-On Web



© 2020 Meta4 Spain, S.A. Se reservan todos los derechos.

AVISO: Este manual está protegido por la legislación referente a propiedad intelectual e industrial y por tratados internacionales. La utilización permitida de esta documentación queda limitada a su uso en conexión con el producto, y todo uso no autorizado será perseguido de acuerdo con la legislación aplicable. Se prohíbe su copia, modificación, reproducción o distribución sin permiso del titular.

Meta4 PeopleNet © 1999 Meta4 Spain, S.A. Se reservan todos los derechos.

Meta4 KnowNet © 1996 Meta4 Spain, S.A. Se reservan todos los derechos.

Meta4 e-mind © 2001 Meta4 Spain, S.A. Se reservan todos los derechos.

Meta4 PeopleNet Ksystem © 2003 Meta4 Spain, S.A. Se reservan todos los derechos.

Meta4 t.innova © 2003 Meta4 Spain, S.A. Se reservan todos los derechos.

Meta4[®], Meta4Mind[®], Meta4 PeopleNet[®], Meta4 KnowNet[®], Meta4 e-mind[®], Meta4 PeopleNet Ksystem[®] y Meta4 t.innova[®] son marcas registradas propiedad de Meta4Spain, S.A.

Otros nombres de compañías, productos o servicios son marcas registradas o nombres comerciales de sus respectivos propietarios.

Meta4 Spain, S.A.
Centro Europa Empresarial
Edificio Roma
C/ Rozabella, 8
Ctra. de La Coruña, km 24,200
28290 Las Rozas, Madrid
ESPAÑA
<http://www.meta4.com>

Fecha de creación: junio de 2007.

Fecha de la última publicación: noviembre de 2020.

Tabla de contenidos

Introducción

■ Acerca de esta guía	1
Propósito	1
Audiencia	1
Requerimientos	1

1 Single Sign-On Web

■ Acerca de este capítulo	3
■ Single Sign-On en aplicaciones Web de Meta4	4
Identificación única del usuario con soporte a dominios de autenticación	5
Definición de alias de usuarios y dominios de autenticación	5
Configuración de las claves	7
Utilización del API de generación de credenciales	8
Librerías necesarias	8
Invocación del API de generación de credenciales	9
Generación del HTML para la conexión con identificación única del usuario	9
Conexión a Web con identificación única del usuario	10
Conexión a Rich Web con identificación única del usuario	10
Identificación única del usuario sin soporte a dominios de autenticación	14
Generador del repositorio de claves	14
Utilización del API del generador de credenciales	15
Librerías necesarias	15
Invocación del API de generación de credenciales	15
Generación del HTML para la conexión con identificación única del usuario	15
■ Conexión con identificación única del usuario mediante SAML	16
Introducción	16
Arquitectura y requisitos	16
Configuración del filtro SSO SAML	17
Creación de dominio indirecto y archivo PKCS12	18
Configuración del filtro en web.xml	18
Verificación de la configuración de configclient.xml	19

Creación de accesos directos	20
------------------------------------	----

Introducción

Acerca de esta guía

Propósito

Esta guía tiene como objetivo proporcionar una descripción de alto nivel de la configuración de la conexión con identificación única del usuario (Single Sign-On - SSO) en entornos Web por sociedad mediante el uso de dominios de autenticación.

Audiencia

Esta guía va dirigida a aquellos desarrolladores que necesitan conocer e implantar la conexión con identificación única del usuario de las aplicaciones Web de Meta4 en entornos multicliente.

Requerimientos

Se asume que el lector tiene conocimientos de:

- La arquitectura de la aplicación Meta4 y la estructura de sus componentes, los Meta4Objects y otros componentes relacionados.

Single Sign-On Web

Acerca de este capítulo

En este capítulo se explican los diferentes procesos para llevar a cabo una conexión con identificación única del usuario entre aplicaciones Web (Single Sign-On Web), siendo una de las aplicaciones de Meta4.

La identificación única de usuario es un esquema que permite que una autenticación previa realizada siguiendo un procedimiento cualquiera no tenga que repetirse para dar acceso a otros entornos informáticos.

En la práctica implica que el usuario no tenga que volver a introducir, por ejemplo, su usuario y contraseña para tener acceso a distintos sistemas.

La identificación única del usuario afecta a los siguientes productos de Meta4:

- Aplicaciones para clientes HTML (como SSE / SSM)
- PeopleNet - Rich Web



El cliente de desarrollo de Meta4 no está incluido en el sistema de conexión con identificación única del usuario aquí propuesto.

Single Sign-On en aplicaciones Web de Meta4

Para implementar un mecanismo de identificación única de usuario, existen distintas tecnologías, cuyas diferencias se detallan a continuación.

1. Identificación única del usuario **entre elementos de Meta4 PeopleNet** mediante certificados generados a partir de un par único de claves de repositorio.
 - Puede consultar las características de este mecanismo de identificación única en la sección *Identificación única del usuario entre entornos de Meta4* del manual *Administración_Aplicaciones_Web.pdf*.
2. Identificación única del usuario **en la conexión con Windows**.
 - Puede consultar las características de este mecanismo de identificación única en el manual *Single_Sign_On_Windows.pdf*.
3. Identificación única del usuario desde un portal Web a Meta4 PeopleNet, mediante el uso de **dominios de autenticación**. Un dominio permite utilizar varios pares de claves de repositorio y varios ficheros PKCS12, y por tanto, sirve para garantizar la seguridad en entornos multicliente.
 - Para más detalle véase la sección *Identificación única del usuario con soporte a dominios de autenticación* del presente manual.
4. Identificación única del usuario desde un portal Web a Meta4 PeopleNet, mediante el uso de un par de claves de repositorio único y un único archivo PKCS12. Este mecanismo garantiza la seguridad en entornos monocliente, empleando una sola clave en el dominio 'LOCAL'. Sin embargo, se recomienda configurar el mecanismo descrito en el punto anterior, que soporta el concepto de "alias".
 - Para más detalle véase la sección *Identificación única del usuario sin soporte a dominios de autenticación* del presente manual.
5. Identificación única del usuario desde un portal Web a Meta4 PeopleNet, mediante **SAML**.
 - Esta integración utiliza la tecnología de **dominios de autenticación**, más un filtro que se encarga de recoger la información proporcionada via SAML. Para más detalle véase la sección *Conexión con identificación única del usuario mediante SAML* del presente manual

Identificación única del usuario con soporte a dominios de autenticación

En este apartado se describen los pasos necesarios para la puesta en marcha de la identificación única del usuario con aplicaciones Web de Meta4 mediante el uso de dominios de autenticación.

Un dominio de autenticación se define como un espacio de nombres en el que los identificadores de usuarios no pueden repetirse. En un entorno multicliente, podría asociarse un dominio de autenticación a cada cliente que quiera establecer identificación única, aunque en un entorno de un solo cliente, podrían utilizarse los dominios de autenticación para establecer espacios de nombres alternativos.

Es necesario realizar dos pasos. El primero, utilizando un cliente de desarrollo, requiere definir alias de usuarios de aplicación, dominios de autenticación, configurar las claves de repositorio y exportarlas a fichero. En el segundo, se realiza un desarrollo en Java para que, cuando el usuario está autenticado en el sistema externo, se garantice el acceso seguro a la aplicación Web de Meta4. Primero se genera una credencial temporal, y luego se realiza una redirección al entorno Web deseado, también mediante la llamada a un código Java.

Definición de alias de usuarios y dominios de autenticación

Para poder conectarse a la aplicación mediante un mecanismo de autenticación externa con Single Sign On, debe establecerse un alias para cada usuario de aplicación.

Así, cuando el usuario presenta una credencial generada con un fichero PKCS12 con dominio, la autenticación se ocupará de buscar al usuario en la tabla de mapeos (M4RSC_USER_ALIAS). Por tanto, es obligatorio que cuando un usuario está haciendo SSO Web en un entorno multidominio exista un mapeo ID_APPUSER | USER_ALIAS | ID_DOMAIN, y solo uno.

Por el contrario, cuando se presente la credencial generada a partir de un fichero PKCS12 sin dominio, se verificará que el usuario que se está presentando no aparezca en la tabla de dominios. Si un usuario presenta un identificador cuyo alias aparece en la tabla de mapeos no se va a poder conectar.

La autenticación también verifica si este es un usuario que tiene restringida la funcionalidad de Single Sign-On en la tabla de usuarios. En caso de que esté restringida, se da un error de autenticación.

EJEMPLO

Supongamos que el ID usuario de aplicación en un sistema origen es charleslupin, mientras que en el sistema destino es CLUPIN.

El usuario se puede conectar como CLUPIN siendo charleslupin en el sistema origen si dispone del fichero PKCS12 emitido para el dominio sirius.meta4.com, por ejemplo:

ID_APPUSER	ID_DOMAIN	USER_ALIAS
charleslupin	sirius.meta4.com	CLUPIN

Si no dispone de un alias para el dominio donde aplica el fichero no podrá conectarse al sistema destino.

Si dispone de más de un alias, para usuarios válidos en fechas en el sistema destino, tampoco podrá conectarse, porque en el sistema destino hay una ambigüedad y la identificación no es posible.

En el caso de tener más de un alias pero que solo uno de los usuarios sea válido o esté fuera de fechas, se da esta situación como válida y se permite el acceso con ese usuario.

Para establecer un alias de usuario, si dispone de un entorno de desarrollo, puede acceder a la opción de menú **Herramientas | Herramientas de desarrollo | Seguridad | Usuarios de Aplicación** y utilizar la sección **Alias de usuario de aplicación** y definir dicho alias a través de los siguientes campos.

- **Alias de usuario aplicación:** alias de mapeo (nombre de usuario definido en el entorno de autenticación Web) utilizado para la integración. Cada usuario debe tener un alias en el dominio de autenticación para poder conectarse a la aplicación cuando se usa un fichero PKCS12 con dominio.
- **ID Dominio:** introduzca en este campo el identificador del dominio de autenticación.

Si no dispone de un entorno de desarrollo, durante la creación de usuarios de aplicación en **Herramientas | Área de Configuración | Gestión de usuarios | Mantenimiento de usuarios**, puede definir los alias a través de los siguientes campos:

- **Usuario SSO Web:** introduzca en este campo el alias de mapeo.
- **Permitir autenticación SSO Web:** esta casilla de verificación permite al usuario conectarse al sistema mediante el mecanismo de autenticación SSO Web.

También durante la creación masiva de usuarios, existe la posibilidad de crear masivamente dichos alias mediante una opción de menú que permite completar

el proceso de creación realizando la importación masiva de alias de usuarios de aplicación para mecanismos de autenticación externa como SSO Web (**Herramientas | Área de Configuración | Gestión de usuarios | Creación masiva de usuarios**).

El proceso de importación no requiere ninguna configuración especial. Sólo debe seguir cuatro sencillos pasos en el orden indicado para ejecutar correctamente las importaciones:

1. Para empezar, debe ejecutar la opción de menú **Generación de plantillas** para obtener el archivo Excel sobre el que se deben añadir los alias para mapear los usuarios. Deberá especificar el tipo de autenticación externa y el campo por el que desea identificar al usuario al realizar la importación (ID Persona o correo electrónico).
2. El Excel resultante tiene las siguientes columnas: Correo electrónico, ID Persona, Usuario LDAP y Usuario SSO Web.
3. A continuación debe añadir en el archivo Excel tantas filas como alias desee importar.
4. Seguidamente, puede ejecutar la importación desde la opción de menú **Importación de datos**. Deberá indicar la ruta al archivo Excel que contiene los alias de usuario. Una vez ejecutada la importación, puede visualizar los resultados de la misma.
5. Por último, puede ejecutar la opción de menú **Histórico de importaciones** para ver todas las importaciones realizadas y los resultados de las mismas.

Configuración de las claves

El primer paso para la puesta en marcha de la integración es la creación de dominios de autenticación. Para crear un nuevo dominio, seleccione la opción de menú **PeopleNet | Herramientas de desarrollo | Configuración de aplicación | Dominios de autenticación | Gestión de dominios de autenticación** y cree un nuevo dominio.

A continuación, cree un par de claves para ese dominio accediendo a **PeopleNet | Herramientas de desarrollo | Configuración de aplicación | Dominios de autenticación | Gestión de claves por dominio** y haciendo clic en **Nuevo**. En la pestaña **Claves por dominio**, rellene los siguientes campos:

- **Id. Certificado:** identificador del certificado
- **Id. Dominio:** elija el identificador del dominio que ha creado
- **Fecha Inicio:** fecha de inicio de validez del certificado.
- **Fecha Fin:** fecha de fin de validez del certificado.
- **Descripción Certificado:** campo descriptivo del certificado.

A la izquierda de la presentación, dispone de dos opciones: **Generar claves de firma** y **Exportar claves a fichero**. Si es la primera vez que accede a esta presentación es recomendable la generación de las claves. Una vez generadas las claves estas ya no deben ser modificadas, o será necesario regenerar el fichero.

Una vez disponibles las claves, pueden ser exportadas a un archivo utilizando la opción de **Exportar claves a fichero**. Se solicitará el valor de los siguientes dos argumentos:

- **AI_FILE_PATH**: ruta para el nuevo repositorio de claves
- **AI_KEY_PASS**: la clave que sella la clave privada en el repositorio.

El fichero generado está en formato PKCS12 y contiene un certificado X.509 con su clave pública y privada de 2048 bits. Estas claves también se quedan almacenadas en el repositorio de Meta4. El fichero es válido únicamente para el dominio de autenticación indicado. Este fichero debe ser transportado al entorno Web de autenticación donde se generarán las credenciales.

Utilización del API de generación de credenciales

Meta4 proporciona el API para la generación de credenciales y acceso a la infraestructura tecnológica. Sin embargo es necesario un desarrollo particular para, entre otras cosas:

- Identificar el usuario que está logado en el portal
- Invocar la infraestructura proporcionada por Meta4.
- Enviar el resultado (HTML) al navegador cliente.

Asimismo, para el funcionamiento aquí descrito, es necesaria la actualización tecnológica de las distintas aplicaciones de Meta4, además de la inclusión y configuración del Generador de Credenciales.

Librerías necesarias

Para el correcto funcionamiento del API de generación de credenciales es necesario que estén las siguientes librerías en el CLASSPATH de Java (según contenedor Web origen)

- m4common.jar, bcbase64.jar, log4j.jar, bcprov-jdk15-146.jar

Estas librerías están disponibles en el CD del producto dentro de la carpeta *m4websso* en la componente *ToolKit*.



Si está utilizando la componente M4WEBSSO (librería m4common.jar), deberá actualizarla manualmente reemplazando los archivos jar existentes en su portal por los nuevos que se

suministran en el CD. Consulte las KBs QA7416 (español) o QA7415 (inglés) para obtener instrucciones más precisas.

Invocación del API de generación de credenciales

La función Java que genera la credencial se llama `GenerateM4Credential()` y pertenece a la clase `com.meta4.common.m4cred.M4Credential`.

```
public String GenerateM4Credential(String ai_sRepositoryPath, String  
ai_sPass, String ai_sUserID, String ai_sLang, long ai_IExpiration, long  
ai_IExpirationMargin)
```

Esta función espera como argumentos:

- `ai_sRepositoryPath`: La ruta al fichero de salida en formato PKCS12
- `ai_sPass`: La clave para proteger la clave privada en el repositorio de claves
- `ai_sUserID`: El usuario que se desea conectar.
- `ai_sLang`: El idioma de conexión
- `ai_IExpiration`: Número de segundos de validez de la credencial. Debería ser lo suficientemente alto como para abarcar el tiempo que puede tardar en realizarse todas las operaciones de transporte, descriptación, o verificación de la credencial (por ejemplo, 600 segundos, 10 minutos).
- `ai_IExpirationMargin`: Número de segundos de margen para desfases temporales entre el servidor Web y el servidor de aplicaciones (por ejemplo, 60 segundos, 1 minuto).

Cuando la credencial llega al servidor de aplicaciones, éste comprueba que ha sido generada para el dominio que se generó el fichero PKCS12, y que el instante actual está entre la fecha de inicio y la fecha de fin, calculados en el servidor Web origen como:

- Fecha de inicio: Instante actual - margen
- Fecha de fin: Instante actual + rango de validez + margen

La función retorna una cadena representativa del certificado codificada en Base64.

Generación del HTML para la conexión con identificación única del usuario

Existen dos posibles productos de destino para la credencial: el cliente Rich Web y el portal para clientes HTML. Para cada uno de ellos existe un HTML específico que permite que el navegador cliente pueda conectarse al producto. Para generar dicho HTML existen dos funciones Java que reciben como

entrada distintos parámetros, entre ellos el certificado generado en el punto anterior. Pasamos a detallar cada una de estas funciones.

Conexión a Web con identificación única del usuario

Función generate()

La función a invocar es el método generate () de la clase com.meta4.common.m4cred.GenerateRedirectHTML:

GenerateRedirectHTML.generate(sCredential, sUsr, sLang, sBaseURL, sServlet, sInitURI, sGotoURI);

Espera como parámetros de entrada:

- La credencial generada para el usuario en la función "GenerateM4Credential".
- El nombre del usuario autenticado en el contenedor Web
- El idioma para la conexión (ej. 2 para inglés, 3 para español, etc)
- La URL base para el servidor Web destino en el que se encuentra instalada la aplicación Web de Meta4 (protocolo, host, puerto)
- La ruta relativa para el servlet de login (normalmente /servlet/login).
- La ruta relativa (URI) para la página de inicialización de la aplicación Web. Esta ruta puede ser nula si no existe ninguna página que deba ser inicializada antes de llamar a la página destino.
- La ruta relativa para la página destino de identificación única del usuario.

Como resultado, esta función devuelve una cadena con el contenido HTML que debe ser enviado al navegador cliente. De ese modo el navegador efectúa una redirección a la aplicación Web.

El mecanismo de redirección no utiliza la tecnología de cookies pero es necesaria para mantener la sesión contra la aplicación Web de destino.

Conexión a Rich Web con identificación única del usuario

Función generate()

La función generate () en la clase com.meta4.common.m4cred.GenerateRedirectPnet se encarga de producir el contenido HTML necesario para entrar en un entorno Rich Web:

GenerateRedirectPnet.generate(credential, userWithDomain, lang, netUpdService, appServerId);

Esta función espera como parámetros de entrada:

-
- La credencial generada para el usuario en la función "GenerateM4Credential".
 - El nombre del usuario autenticado en el contenedor Web.
 - El idioma de conexión (ej. 2 para inglés, 3 para español, etc.).
 - La URL base para el servidor Web en el que se encuentra el servicio de actualizaciones (protocolo, host, puerto). Así, por ejemplo, si la URL de entrada al Rich es: `http://maquina:puerto/m4updateservices/m4richweb.html`, en este parámetro se debe indicar tan solo la siguiente cadena: `http://maquina:puerto/m4updateservices/`.
 - El servidor de aplicaciones con el que se establecerá la conexión.

Como resultado, esta función devuelve una cadena con el contenido HTML que debe ser enviado al navegador cliente. Como resultado el navegador efectúa una redirección a la aplicación Rich Web.

EJEMPLO

```
String sResultRedirect = GenerateRedirectPnet.generate(sCredential,
sUserID, sLangID, sUpdateServicesURI, sServerID);
```

Función generateExtended()

La función `generateExtended()` en la clase `com.meta4.common.m4cred.GenerateRedirectPnet` se encarga de producir el contenido HTML necesario para entrar en un entorno Rich Web, permitiendo un comportamiento más detallado para el comportamiento de conexión:

```
GenerateRedirectPnet.generateExtended(sCredential, sUserID, sLangID,
sUpdateServicesURI, sServerID, oExtendedParams);
```

Esta función espera como parámetros de entrada:

- La credencial generada para el usuario en la función "GenerateM4Credential".
- El nombre del usuario autenticado en el contenedor Web.
- El idioma para la conexión (ej. 2 para inglés, 3 para español, etc.).
- La URL base para el servidor Web en el que se encuentra el servicio de actualizaciones (protocolo, host, puerto). Así, por ejemplo, si la URL de entrada al Rich comienza por `http://maquina:puerto/m4updateservices`, en este parámetro se debe indicar tan solo la siguiente cadena: `http://maquina:puerto/m4updateservices/`.

- El identificador del servidor al cual se realizará la petición, según la configuración de los servicios de actualización.
- Y un objeto de parámetros extendidos de tipo `ExtendedParamsRich`. La clase `ExtendedParamsRich` contiene un método `setLogonConfig` que acepta valores particulares de la clase `LogonConfigRich`, que son los siguientes:
 - `NONE`: indica a la función no ejecutar ninguna acción.
 - `ROLE_AND_PARAMS`: muestra la ventana de selección de roles y de configuración de parámetros de sesión.
 - `ROLE_ONLY`: muestra solamente la ventana de selección de roles.
 - `PARAMS_ONLY`: muestra solamente la ventana de configuración de parámetros de sesión.

EJEMPLO

```
String sUserID = "user@meta4.com";
String sLangID = "3";
String sUpdateServicesURI = "http://hodor.meta4.com:8010/
m4updateservices";
String sServerID = "SQLPRU4";

String sResultRedirect = "";
ExtendedParamsRich oExtendedParams = new ExtendedParamsRich();
oExtendedParams.setLogonConfig(LogonConfigRich.NONE);
sResultRedirect =
GenerateRedirectPnet.generateExtended(sCredential, sUserID,
sLangID, sUpdateServicesURI, sServerID, oExtendedParams);

String sResultRedirect = "";
ExtendedParamsRich oExtendedParams = new ExtendedParamsRich();
oExtendedParams.setLogonConfig(LogonConfigRich.ROLE_AND_PARAMS);

ExtendedParamsRich oExtendedParams = new ExtendedParamsRich();
oExtendedParams.setLogonConfig(LogonConfigRich.ROLE_ONLY);
sResultRedirect =
GenerateRedirectPnet.generateExtended(sCredential, sUserID,
sLangID, sUpdateServicesURI, sServerID, oExtendedParams);

ExtendedParamsRich oExtendedParams = new ExtendedParamsRich();
oExtendedParams.setLogonConfig(LogonConfigRich.PARAMS_ONLY);
```

```
sResultRedirect =  
GenerateRedirectPnet.generateExtended(sCredential, sUserID,  
sLangID, sUpdateServicesURI, sServerID, oExtendedParams);
```

*Donde **sCredential** debe obtenerse mediante las funciones indicadas en secciones anteriores.*

La llamada al código Java para las conexiones con identificación única de usuario en entornos multidominio es la misma que en los entornos sin dominios de autenticación. Sin embargo, internamente el código Java sabe si el archivo PKCS12 ha sido generado con o sin dominio de autenticación. El certificado es diferente para cada sociedad, para que puedan coexistir distintas sociedades.

Identificación única del usuario sin soporte a dominios de autenticación

En este apartado se describen los pasos necesarios para la puesta en marcha de la componente de integración de la conexión Web con identificación única del usuario de Meta4 sin dominios de autenticación.



Esta configuración se aplica solamente a los usuarios conectados con rol de aplicación M4ADM.

Generador del repositorio de claves

En el menú seleccione **Herramientas | Configuración de aplicación | Gestión de repositorio de claves**.

Se presentan dos opciones: **Generar claves (pública/privada) de firma** y **Exportar claves a fichero**. Si es la primera vez que accede a esta presentación, es recomendable la generación de claves. Una vez generadas las claves, ya no deben ser modificadas, ya que ello imposibilitaría la conexión de los clientes que actualmente estén utilizando la identificación única del usuario entre distintos repositorios de Meta4.



Tenga en cuenta que, aunque desde la opción **Gestión de repositorio de claves** es posible crear varios registros, el único registro utilizable a día de hoy sin dominios de autenticación es el contenido en el estándar con identificador "Meta4 Server Site = LOCAL".

Una vez disponibles las claves, estas pueden ser exportadas utilizando la opción **Exportar claves a fichero**. Se solicitarán 2 parámetros:

- Ruta para el nuevo repositorio de claves
- La clave que sella la clave privada en el repositorio.

El fichero generado está en formato PKCS12 y contiene un certificado X.509 con su clave pública y privada de 1024 bits. Estas claves también se quedan almacenadas en el repositorio de Meta4. Este fichero debe ser transportado al entorno Web de autenticación donde se generarán las credenciales.

Utilización del API del generador de credenciales

Librerías necesarias

Para el correcto funcionamiento del API de generación de credenciales es necesario que estén las siguientes librerías en el CLASSPATH de Java (según contenedor Web origen)

- m4common.jar, bcbase64.jar, log4j.jar, bcprov-jdk15-146.jar

Estas librerías están disponibles en el CD del producto dentro de la carpeta *m4websso* en la componente *ToolKit*.



Si está utilizando la componente M4WEBSSO (librería m4common.jar), deberá actualizarla manualmente reemplazando los archivos jar existentes en su portal por los nuevos que se suministran en el CD. Consulte las KBs QA7416 (español) o QA7415 (inglés) para obtener instrucciones más precisas.

Invocación del API de generación de credenciales

La función Java que genera se llama `GenerateM4Credential()` y pertenece a la clase `com.meta4.common.m4cred.M4Credential`. Esta función espera como argumentos:

- La ruta al fichero de salida en formato PKCS12
- La clave para proteger la clave privada en el repositorio de claves
- El usuario para el cual se generará el certificado
- El idioma de conexión (reservado para uso futuro)
- El tiempo de expiración y el margen de error de sincronización de relojes entre la máquina que genera el certificado y la máquina donde se ejecutará el servidor de aplicaciones de Meta4 destino.

La función retorna una cadena representativa del certificado codificada en Base64.

Generación del HTML para la conexión con identificación única del usuario

El procedimiento y las funciones Java para generar el HTML específico que permite que el navegador cliente pueda conectarse al producto son los mismos que en el caso de la identificación única del usuario con soporte a dominios de autenticación. Consulte el procedimiento en dicha sección.

Conexión con identificación única del usuario mediante SAML

Introducción

En esta sección se describe la integración del protocolo de identificación única del usuario desarrollado por Meta4 con el protocolo estándar SAML (Security Assertion Markup Language). La integración con SAML 2.0 permite evitar la instalación local de la tecnología de Meta4 en el entorno del cliente, que no tiene que alojar el fichero PKCS12 en su entorno local, ya que los archivos necesarios quedan instalados en el propio entorno de Meta4.

La integración con SAML se basa en la tecnología de dominios de autenticación detallada en el apartado anterior. Para permitir la identificación única del usuario con el estándar SAML 2.0 dentro de las aplicaciones de Meta4, se crea un dominio de uso interno, al que se conoce como 'dominio indirecto'.

Tenga en cuenta que antes de poder iniciar el proceso de configuración de la integración con SAML 2.0, es preciso instalar el entorno de autenticación SAML 2.0 Shibboleth.

Arquitectura y requisitos

Dada una topología de aplicaciones Meta4 en la que se dispone de un portal (aplicación Web para clientes HTML) y un servidor de actualización para clientes Rich Web, para permitir la identificación única del usuario basada en SAML, se necesitan unas componentes Java que se instalan automáticamente junto con las componentes tecnológicas del portal (cliente ligero).

Antes de configurar dichas componentes, a las que nos referiremos como filtro SSO SAML, hay que tener en cuenta que la topología debe estar sujeta a las siguientes restricciones:

1. El acceso se debe realizar por un punto único: el portal.

Si para acceder al portal, por ejemplo, el usuario teclea: `https://xxx.meta4globalhr.com`, para iniciar sesión con Rich Web, debe teclear una URL que se origine en la del portal, por ejemplo: `https://xxx.meta4globalhr.com/rw`. A la URI concatenada al portal para obtener los servicios de actualización la llamaremos URI de redirección, por ejemplo, `/rw`.

La URI de redirección se puede configurar de varias formas:

-
- Creando un directorio virtual para los servicios de actualización dentro de la instalación del portal de cliente ligero.
 - Creando una redirección a otro servidor donde se encuentren los servicios de actualización <https://updateservices.meta4globalhr.com>

2. El nombre de host de acceso al portal distingue ámbitos de autenticación.

En los entornos en los que múltiples clientes pueden acceder al portal de cliente ligero y/o los servicios de actualización, se debe hacer que el nombre con el que accede cada cliente distinto sea diferente.

El motivo es que, en el ámbito SAML, el cliente se identifica por el nombre de Host con que accede al servidor Web (cabecera Host). Es decir, si hay dos clientes distintos, cliente1 y cliente2, que acceden al mismo servidor Web, el administrador del sistema debe configurar el acceso al servidor para que ambos utilicen una URL distinta, por ejemplo: <https://customer1.meta4globalhr.com> y, <https://customer2.meta4globalhr.com>

El acceso personalizado se puede crear de formas distintas, la principal es crear un host virtual para cada cliente que accede a la plataforma.

3. El agente Shibboleth debe estar configurado con el idP del cliente.

El proveedor de identidad (idP) de cliente debe ser configurado a partir de los metadatos de cada cliente añadido. El proveedor de identidad debe ser compatible con SAML v2. Estos metadatos pueden ser obtenidos por el administrador del sistema a partir de la instalación de Shibboleth tecleando la URL <http://localhost/shibboleth.sso/Metadata>.

Para cada cliente se debe configurar el archivo shibboleth2.xml para añadirle las referencias a los metadatos del proveedor de identidad del cliente.

Configuración del filtro SSO SAML

Los pasos necesarios para configurar el filtro de SSO SAML son los siguientes

1. Crear el dominio indirecto y el archivo PKCS12 de dicho dominio.
2. Configuración del filtro en web.xml.
3. Verificar la configuración de configclient.xml.
4. Crear accesos directos por idioma.

Creación de dominio indirecto y archivo PKCS12

Es necesario crear un archivo PKCS12 que servirá para todos los clientes que estén previamente autenticados por un mecanismo tipo SAML. Este archivo residirá en el servidor Web del cliente ligero.

- Creación del dominio de autenticación indirecto: cree un dominio para la autenticación con SSO SAML desde la opción de menú **Gestión de dominios de autenticación** situada en **PeopleNet | Herramientas de desarrollo | Configuración de aplicación | Dominios de autenticación**. Este dominio debe llamarse “WEBSSO_INDIRECT_DOMAIN”.
- Generación del archivo de claves del dominio de autenticación: ejecute la opción de menú **PeopleNet | Herramientas de desarrollo | Configuración de aplicación | Dominios de autenticación | Gestión de claves por dominio** y seleccione la acción **Exportar claves a fichero** (al que nos referiremos como ca-saml.p12). Anote la contraseña con que lo ha generado (por ejemplo changeit). Ubique el archivo en una ruta accesible al servidor Web del SSE/SSM y anote la ruta.

Ejemplo: ruta c:\temp\ca-saml.p12 y contraseña changeit.

Configuración del filtro en web.xml

Active el filtro de identificación única del usuario editando el archivo web.xml en la ruta <raíz-app>/WEB-INF/web.xml, y cambiando el parámetro enable= true como se indica en el siguiente ejemplo:

EJEMPLO

```
<filter>
  <filter-name>M4WebSSOFilter</filter-name>
  <filter-class>com.meta4.filters.websso.M4WebSSOFilter</filter-
class>
  <init-param>
    <param-name>enable</param-name>
    <param-value>true</param-value>
  </init-param>
  <init-param>
    <param-name>richwebpattern</param-name>
    <param-value>^/rw</param-value>
  </init-param>
  <init-param>
    <param-name>esspattern</param-name>
```

```
<param-value>/</param-value>
</init-param>
</filter>
```

Los parámetros `richwebpattern` y `esspattern`, respectivamente, indican los patrones (expresiones regulares) que determinan cuándo una petición es para los servicios de actualización y cuándo para el portal.

Verificación de la configuración de `configclient.xml`

En el servidor Web en que se encuentra el portal de cliente ligero, abra el fichero `<raiz-app>/WEB-INF/classes/properties/configclient.xml`, para configurar las siguientes entradas:

```
<filepath>c:/temp/ca-saml.p12</filepath>
<keypass>changeit</keypass>
<updatesurl>https://www.meta4globalhr.com/rw</updatesurl>
<richappserver>localhost</richappserver>
<interval>600</interval>
<margin>60</margin>
<portalprotocol>https</portalprotocol>
<portalport>443</portalport>
<portallogonpattern>generico_login.jsp</portallogonpattern>
<portaldefaulturi>/servlet/CheckSecurity/JSP/sse_generico/
ssco_portal.jsp</portaldefaulturi>
```

Donde, para el caso general, se tiene:

- **filepath**: ruta al certificado del dominio indirecto `ca-saml.p12`
- **keypass**: la contraseña que protege el acceso al certificado
- **interval**: intervalo de duración de la credencial interna
- **margin**: margen de error para la comprobación de la credencial interna

Mientras que para el caso de acceso por el portal:

- **portallogonpattern**: patrón que identifica la página de conexión. Cada vez que un usuario al que se reconozca como con autenticación SAML, introduzca una URL que contenga este patrón, se le redirigirá, bien a la página por defecto, o bien a la página a la que iba en el caso de ser un enlace directo recibido por e-mail.
- **portalprotocol**: protocolo de la URL a la que redirigir en caso de que el usuario introduzca la página de conexión.

- **portalport**: puerto de la URL a la que redirigir en caso de que el usuario introduzca la página de conexión.
- **portaldefaulturi**: página a la que se redigirá en caso de que el usuario ya estuviera conectado, y por inactividad, se le estuviera redirigiendo a la página de conexión.

Y el caso del acceso Rich Web:

- **updatesurl**: ruta en la que está disponible la URL de actualización.
- **richappserver**: servidor de aplicaciones que atiende las peticiones de autenticación. Debe coincidir con la cadena que aparece en la clave "Server" del archivo M4WizardIn.reg (exactamente igual).

EJEMPLO

```
[HKEY_LOCAL_MACHINE\SOFTWARE\Meta4\PeopleNet\CLIENT\Servers\RICHWEB]
"Server" = "tpbasexp.meta4.com"
"IdPort" = "3002"
"AdminPort" = "3004"
"IdSSLPort" = "3003"
"AdminSSLPort" = "3005"
```

Creación de accesos directos

Para que los usuarios accedan desde el sistema externo, habrá que crear accesos directos. Al no haber una forma estándar de establecer una correspondencia entre el idioma en el el sistema externo y en PeopleNet, el idioma es un parámetro de dicho acceso directo

- Usuario del portal Web en español:
<https://customer.meta4globalhr.com/?langid=3>
- Usuario del portal Web en español que va a página concreta:
<https://customer.meta4globalhr.com/servlet/CheckSecurity/JSP/tarea/pagina.jsp?langid=3>
- Usuario del Rich Web en español
<https://customer.meta4globalhr.com/hr?langid=3>
- Usuario del Rich Web por optimizador en <http://websync.customer.com>:
<https://customer.meta4globalhr.com/hr?langid=3&updservice=http://websync.customer.com>